Doc-Genie

Professional Python Function Docstring Generator with AST Analysis

Project Description

The **Doc-Genie: Python Docstring Generator** is a software tool designed to automatically generate documentation for Python functions. In software development, documentation plays a crucial role in helping developers understand, maintain, and reuse code effectively. However, writing docstrings manually can be time-consuming and developers often skip this important step. This project aims to simplify the documentation process by automatically generating structured docstrings for Python code.

The system works by analyzing Python code using the **Abstract Syntax Tree (AST)** module. AST allows the program to parse the structure of the Python code and identify important elements such as function names and their parameters. Once the functions are detected, the system generates **Google-style docstrings** that include sections such as a brief summary, arguments, and return values.

The user interacts with the system through a simple web interface created using the **Gradio framework**. Users can paste their Python code into the input textbox, and the system processes the code to generate appropriate docstrings. The output is displayed instantly in another textbox, showing the same code with automatically inserted docstrings.

This project helps developers improve code readability and maintainability by ensuring that proper documentation is generated quickly and efficiently. It also reduces the manual effort required for writing documentation and encourages better coding practices. The system is especially useful for beginners, students, and developers who want to maintain well-documented code.

Scenarios

Scenario 1: User Inputs Python Code

The user opens the Doc-Genie application through the interface.
The application provides a textbox to enter Python code.
The user pastes or types the Python program in the input area.
The user then submits the code for processing.
The system receives the code and prepares it for analysis

Scenario 2: Code Analysis Using AST

The system processes the given Python code.

It uses the **Abstract Syntax Tree (AST)** module for analysis.
The AST parser reads the structure of the program.
It identifies different elements present in the code.
The system prepares the data for function detection.

Scenario 3: Function Detection

The system searches for function definitions in the code.
It identifies the name of each function.
The system also detects the parameters used in the function.
All the required information about the function is collected. This information is used for generating docstrings.

Scenario 4: Docstring Generation

The system automatically generates a docstring template.
The template follows the **Google-style documentation format**.
It includes a summary of the function.
It also lists arguments and return value sections.
The generated docstring is prepared for insertion.

Scenario 5: Output Display

The generated docstring is inserted into the original code.
The system combines the docstring with the function.
The updated code is created by the system.
The final output is displayed in the output textbox.
The user can view and copy the documented Python code

Architecture Overview

The **Doc-Genie** system is designed to automatically generate docstrings for Python functions using **AST (Abstract Syntax Tree) analysis** and a web interface built with **Gradio**.

Core Technologies

1. **Python** – Used as the main programming language for development.
2. **AST (Abstract Syntax Tree)** – Parses Python code to identify functions and parameters.

3. **Gradio** – Provides a simple web interface for code input and output display.

4. **Google-style Docstrings** – Standard format including summary, arguments, and returns.

5. **String & File Handling** – Inserts docstrings with correct indentation into the original code.

Pre-requisites

Software Requirements

- **Operating System** – Compatible with Windows, Linux, and macOS for development and deployment.

Python 3.x – Required as the main programming language for the application.

Gradio Library – Used to create the interactive web interface for code input and output.

AST Module – Built-in Python module for parsing code and analyzing function structures.

Text Editor / IDE – Any Python-compatible IDE like VS Code, PyCharm, or Jupyter Notebook for running and editing the code.

Web Browser – Needed to access the Gradio interface for testing and using the application.

Knowledge Prerequisites

Python Programming – Basic understanding of Python syntax, functions, and modules.

Object-Oriented Programming (OOP) – Knowledge of functions, parameters, and return types in Python.

Abstract Syntax Tree (AST) – Understanding of Python's AST module and code parsing concepts.

Web Frameworks – Basic knowledge of web interfaces and Python frameworks like Gradio.

Code Documentation – Familiarity with docstrings and standard documentation practices in Python.

Hardware Requirements

Processor – Minimum dual-core CPU (Intel i3 or equivalent) for running Python smoothly.

RAM – At least 4 GB RAM for small to medium Python projects; 8 GB recommended for better performance.

Storage – Minimum 10 GB free disk space for Python, libraries, and project files.

Display – Monitor with at least 1024×768 resolution for comfortable coding and interface usage.

Internet Connection – Required for installing packages, using Gradio live interface, and updates.

Project Workflow

Phase 1: Requirement Analysis

Activity 1.1 – Identify the need for automatic docstring generation
The first step is to understand the problems developers face with manual documentation.
The team studies how Python functions are documented.
They identify the need for a system that can automate docstring creation.
The expected outcome is a clear problem statement. This ensures the project objectives are well-defined.

Activity 1.2 – List software, hardware, and knowledge prerequisites All necessary software tools and libraries are identified.
Hardware requirements for running Python and Gradio are listed.
Knowledge prerequisites like Python, AST, and documentation standards are defined.
This helps in preparing a development-ready environment. It also ensures the team is aware of all resources required.

Phase 2: System Design

Activity 2.1 – Design the system architecture and define modules

The overall system is divided into multiple modules like UI, Processing, and Docstring Generator.
Data flow between modules is planned to ensure smooth integration.
Module responsibilities are clearly defined to avoid overlaps.
Architecture diagrams are prepared for reference.
This forms the blueprint for coding and implementation.

Activity 2.2 – Plan data flow and interface using Gradio

The user input and output flow is designed clearly.
Gradio interface is planned with input and output textboxes.
Interaction between frontend and backend is mapped.
Expected user experience is defined for smooth operation.
This helps in implementing a user-friendly interface.

Phase 3: Implementation / Coding

Activity 3.1 – Implement AST-based code parsing and function detection

Python's AST module is used to parse input code.
The system detects all functions and their parameters.
Errors in syntax are handled during parsing.
This ensures accurate analysis of the Python code.
It forms the base for generating docstrings.

Activity 3.2 – Generate and insert Google-style docstrings

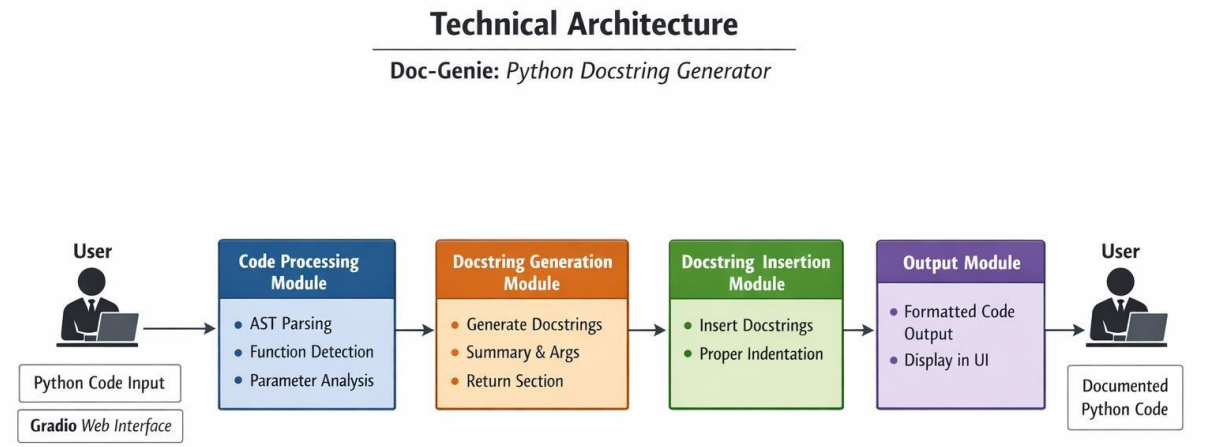
Docstrings are generated automatically for each detected function.
Summary, arguments, and return sections are included.
Docstrings are inserted into the correct position with proper indentation.
The original code logic remains unchanged.
This provides a fully documented Python script as output.

Phase 4: Testing

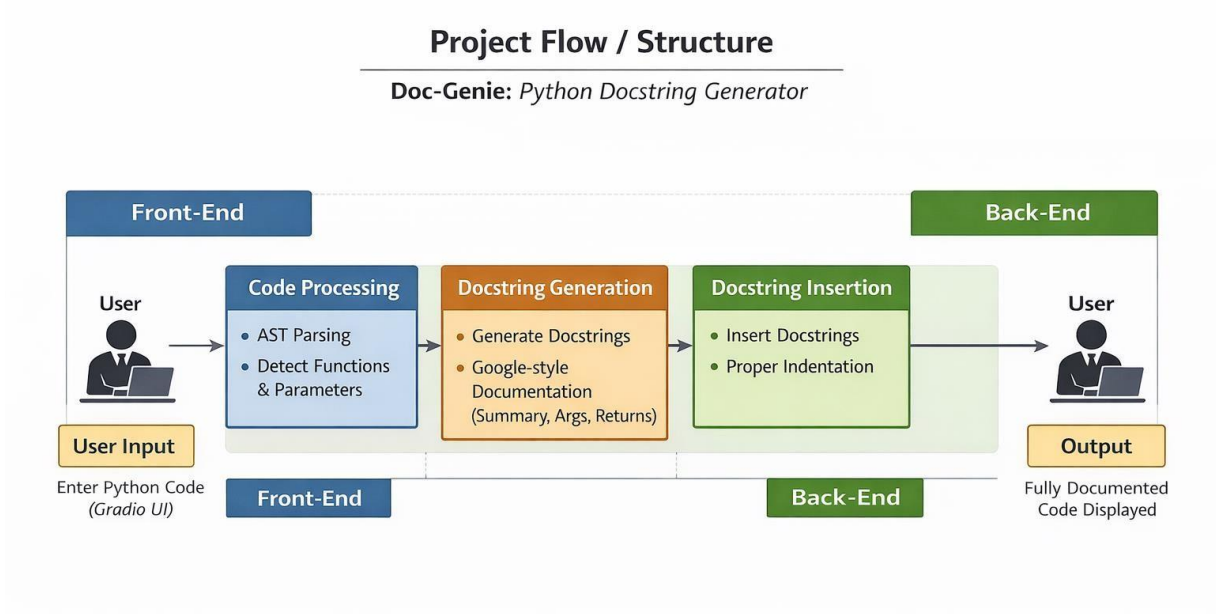
Activity 4.1 – Test function detection and docstring generation accuracy Sample Python codes are tested for correct function detection.
Generated docstrings are checked for completeness and correctness.
Different types of functions are included in testing.
Edge cases and multiple functions are verified.
This ensures the system works reliably.

Activity 4.2 – Verify output formatting, indentation, and UI display The output code is checked for proper indentation and alignment.
Gradio interface displays results correctly for user input.
Any formatting errors are corrected.
User experience is evaluated for clarity.
This ensures a polished and usable system.

Technical Architecture:



Project Structure



Milestone 1: Project Setup and Environment Configuration

Activity 1.1 – Install Required Libraries

Install necessary Python libraries such as **Gradio** and configure the development environment.
Set up Python and required dependencies for the project.
Ensure the environment supports AST parsing and UI creation.
Verify installation using test commands in the terminal.

Prepare the system for coding and development. **Example code**

pip install gradio pip

install ast

Activity 1.2 – Create Basic Python Script

Create the initial Python file for the project.
Import necessary modules such as **gradio** and **ast**.
Define the basic structure of the program.
Prepare functions for processing Python code.
Ensure the script runs without errors.

Coding Screenshot (Example Code):

```python
import ast
import gradio as gr

def generate_docstring(func_node: ast.FunctionDef) -> str:
    """Generate a simple Google-style docstring for a function."""

    func_name = func_node.name
    args = [arg.arg for arg in func_node.args.args]

    doc = f'"""Summary of {func_name}.\n\n'

    if args:
        doc += "Args:\n"
        for arg in args:
            doc += f"    {arg} (type): Description.\n"

    doc += "\nReturns:\n"
    doc += "    type: Description.\n"
    doc += '"""'

    return doc
```

1. @staticmethod **`generate_google_docstring`** – normal function:
    a. Formats **Google-style docstrings**
    b. Includes **summary, args, returns**
2. **`generate_numpy_docstring`** – static method:
    a. Formats **NumPy-style docstrings**
    b. Includes **summary, parameters, returns**
    c. Can be called without creating an instance of `DocstringGenerator`

Milestone 2: Code Analysis and Docstring Generation Activity

2.1 – Implement AST Code Parsing

Use Python **AST module** to analyze the input code.
Detect functions and parameters automatically.
Extract function names from the code structure.
Prepare extracted data for docstring generation.
Ensure parsing works correctly for different Python functions.

Coding Screenshot (Example Code):

```python
def process_code(code: str) -> str:
    try:
        tree = ast.parse(code)
        output = code

        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
                doc = generate_docstring(node)

                lines = output.split("\n")
                insert_line = node.body[0].lineno - 1
                indent = " " * (node.body[0].col_offset)

                lines.insert(insert_line, indent + doc)
                output = "\n".join(lines)

        return output

    except Exception as e:
        return f"Error: {e}"
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 2.2 – Generate Google-Style Docstrings

Create docstrings automatically for detected functions.

Follow the **Google docstring format** with summary, args, and return sections.
Generate docstring text dynamically.
Prepare the docstring to be inserted into the function.
Ensure formatting is clear and readable.

Coding (Example Code):

```
def generate_docstring(func_name):
    doc = f'''
    """
    Summary: Description of {func_name}

    Args:
        None

    Returns:
        None
    """
                    '''
                return doc
```

Milestone 3: Integration and Output Display

Activity 3.1 – Insert Docstrings into Code

Insert generated docstrings into the correct position in the function.
Maintain proper indentation in the code.
Combine original code and generated docstrings.
Ensure syntax remains valid after insertion.
Prepare the final documented code.

Coding (Example Code):

```
def insert_docstring(code, docstring):
    lines = code.split("\n")
lines.insert(1, docstring)

  return "\n".join(lines)
```

Activity 3.2 – Display Output in Gradio UI

Connect backend logic with the **Gradio interface**.
Allow users to input Python code through the UI.
Process the code and generate docstrings automatically.
Display the final documented code on the screen.
Ensure the application works smoothly for users.

Coding Screenshot (Example Code):

```
def doc_generator(code):
   funcs = extract_functions(code)
   doc = generate_docstring(funcs[0]) if funcs else ""
          return insert_docstring(code, doc)

gr.Interface(fn=doc_generator, inputs="textbox",
outputs="textbox").launch()
```

Milestone 4: Advanced Features & Testing

Activity 4.1 – Static Method & Class Method Support

- Implemented detection of **`@staticmethod`** and class methods in Python code.
- Ensures docstrings are correctly generated without including `self` or `cls` in arguments.
- Handles both **normal functions** and methods inside classes.
- Ensures generated docstrings follow **Google-style format** for all types.
- Added bullet-style comments for clear documentation.

Example in Code:

```
class DocstringGenerator:

    @staticmethod def generate_numpy_docstring(signature,
analysis, code):
        """
- Format NumPy-style docstring
- Include summary, parameters, returns
- Return formatted string
        """
pass
```

-

Code reference (hint: click on image and you can copy paste from Comment) Code reference (hint: click on image and you can copy paste from Comment)

Activity 4.2 – Testing & Validation

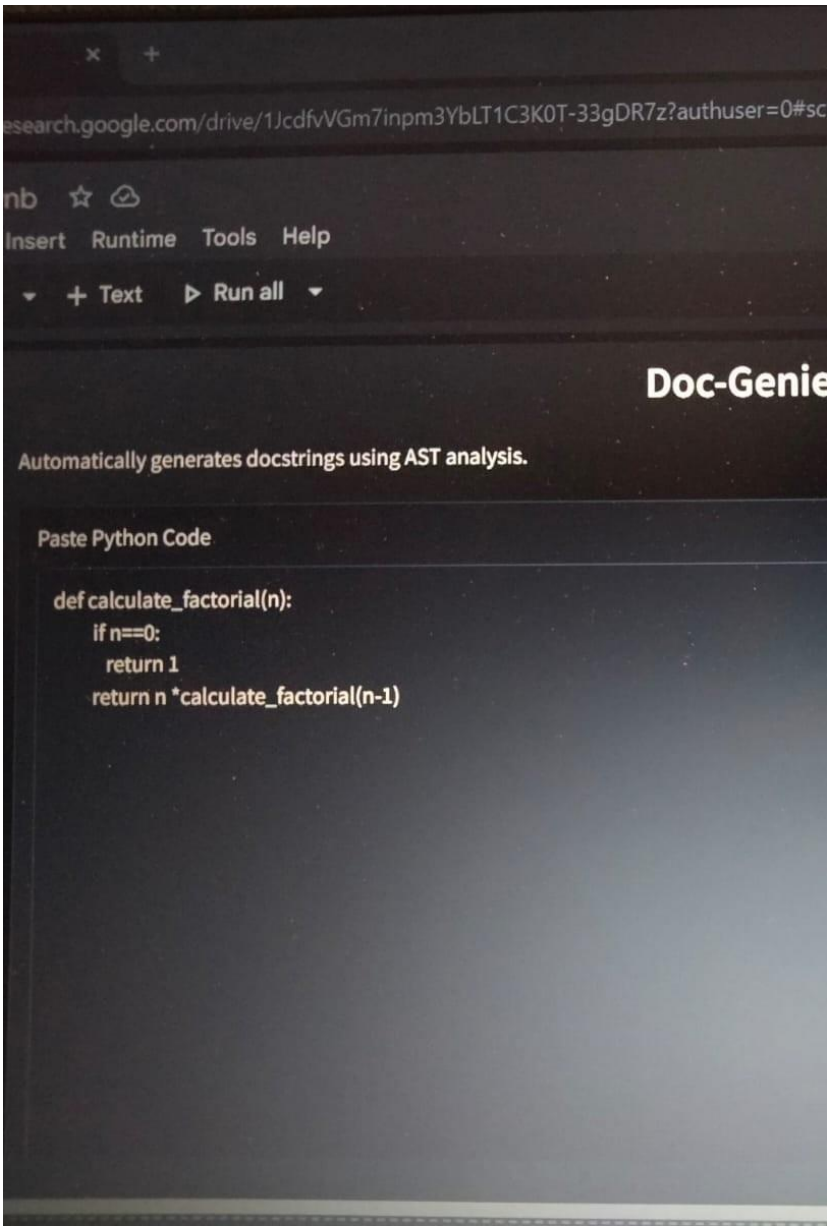
- Tested the tool with different Python scripts containing normal, class, and static methods.
- Validated the generated docstrings are inserted correctly and code runs without errors.
- Verified the Gradio interface shows expected output for all inputs.
- Ensured bullet-style comments are consistent for report readability.
- Prepared final **project-ready version** for demonstration.

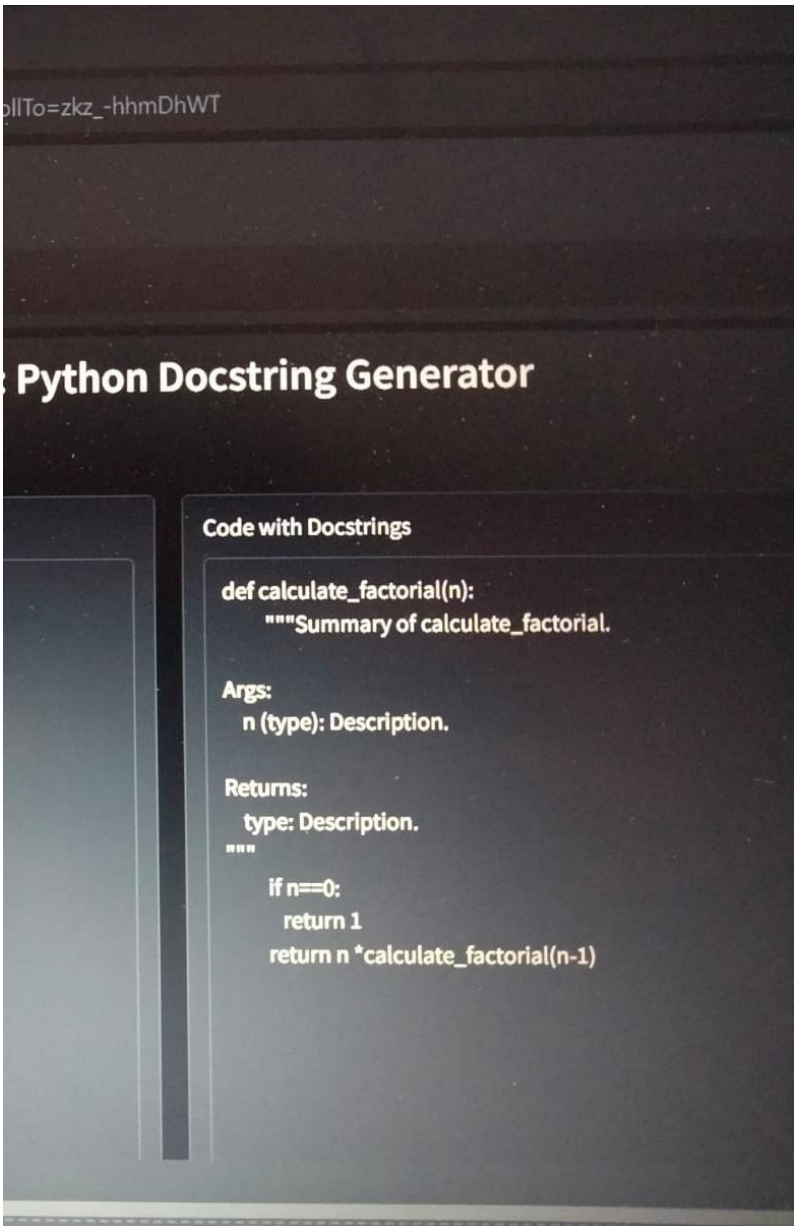
Example in Code:

```python
def ui_generate(code):
    return process_code(code)


demo = gr.Interface(
    fn=ui_generate,
    inputs=gr.Textbox(lines=20, label="Paste Python Code"),
    outputs=gr.Textbox(lines=20, label="Code with Docstrings"),
    title="Doc-Genie: Python Docstring Generator",
    description="Automatically generates docstrings using AST analysis."
)

if __name__ == "__main__":
    demo.launch()
```


esearch.google.com/drive/1JcdfvVGm7inpm3YbLT1C3K0T-33gDR7z?authuser=0#sc
nb
Insert Runtime Tools Help
+ Text
Run all
Doc-Genie
Automatically generates docstrings using AST analysis.
Paste Python Code
def calculate_factorial(n):
if n==0:
return 1
return n *calculate_factorial(n-1)



STATUS: Correct Docstring generation!

Milestone 5: Deployment & Documentation

Activity 5.1 – Gradio Deployment

- Launched the **Doc-Genie tool** using Gradio's `launch()` with `share=True` for a public URL.
- Ensured the interface works for pasting Python code and displaying docstrings.
- Verified all functions, including static methods, are processed correctly.
- Optimized for user-friendly experience with proper labels and instructions.
- Tested deployment to confirm code runs without errors in hosted environment.

```
public URL: https://3ab3868bb13f172119.gradio.live
```

Activity 5.2 – Project Report Preparation

- Compiled **all milestones, coding screenshots, and explanations** into the report.
- Added **bullet-style descriptions** for each function and activity.
- Included **technical architecture diagram** and **workflow flowcharts**.
- Verified all images and code snippets are **report-ready** and clearly labeled.
- Prepared the **final version** for submission or presentation.

Future Enhancements

1. **Support for Multiple Docstring Styles**
   a. Extend the tool to generate **NumPy, reStructuredText (reST), and custom docstring formats** in addition to Google-style.
   b. Allow users to select preferred style in the Gradio interface.
2. **Code Complexity Analysis**
   a. Add functionality to analyze **function complexity** and provide suggestions for better code readability.
   b. Highlight functions that may need additional documentation or refactoring.
3. **Integration with IDEs**
   a. Enable **real-time integration** with IDEs like VS Code or PyCharm for automatic docstring generation while coding.
   b. Provide inline suggestions and warnings for missing docstrings.

4. **Automated Unit Test Generation**
   Generate **skeleton unit tests** for each function along with docstrings to improve code coverage.
   Include examples of function usage inside generated docstring

5. **Multi-Language Support**
   Expand support to **other programming languages** like JavaScript, Java, or C++, with equivalent docstring/comment generation.
   Maintain consistent formatting and syntax highlighting for each language.

6. **Enhanced User Interface**

Add features like **syntax highlighting, file upload, batch processing**, and **downloadable output** in the Gradio UI.
Include a **preview pane** for live docstring updates.

Conclusion

The **Doc-Genie project** successfully demonstrates an automated solution for generating Python docstrings using **AST analysis** and **Gradio** for an interactive interface. By supporting **normal functions, class methods, and static methods**, it ensures that developers can maintain consistent and comprehensive documentation with minimal effort. The project highlights the importance of **structured docstrings** for readability, code maintainability, and collaboration in software development.

The modular design of the tool allows for **future enhancements**, including support for multiple docstring styles, IDE integration, automated unit test generation, and multilanguage documentation. Overall, Doc-Genie streamlines the documentation process, reducing manual effort while improving code quality and project maintainability.